

Tarea 1 Sistemas Distribuidos

Plataforma de Análisis de Consultas con Caché

Equipo T1 Sistemas Distribuidos

Profesor: Nicolás Hidalgo

Abril, 2026

Índice

1. Introducción	2
2. Diseño de la Arquitectura	2
2.1. Descripción General	2
2.2. Componentes Principales	2
2.2.1. Generador de Tráfico	2
2.2.2. Servicio de Caché	2
2.2.3. Generador de Respuestas	2
2.2.4. Métricas	3
3. Dataset y Zonas de Consulta	3
3.1. Google Open Buildings v3	3
3.2. Descarga automática y persistencia	3
3.3. Zonas Geográficas Predefinidas	3
4. Implementación	4
5. Experimentos y resultados	4
5.1. Metodología experimental	4
5.2. Impacto de la distribución de tráfico	5
5.3. Efecto de la política de remoción	5
5.4. Efecto del tamaño de caché	6
5.5. Efecto de TTL	6
6. Discusión	6
6.1. Preguntas del Enunciado	6

7. Conclusiones	7
8. Referencias	7

1. Introducción

Se implementó una arquitectura distribuida compuesta por generador de tráfico, servicio de caché, generador de respuestas y almacenamiento de métricas. El sistema opera sobre el dataset **Google Open Buildings v3** (tile S2 level 4, archivo `967_buildings.csv.gz`), que es descargado automáticamente desde Google Cloud Storage al iniciar el servicio de respuestas. Los datos se precargan en memoria y no se depende de una base de datos externa en tiempo de ejecución. En esta versión del informe se incorporan **resultados experimentales empíricos**, obtenidos ejecutando campañas controladas sobre el despliegue de Docker Compose. También se realizaron **pruebas reales manuales para cada tipo de consulta (Q1–Q5)** que confirmaron la correcta resolución de todas las métricas exigidas.

2. Diseño de la Arquitectura

2.1. Descripción General

La solución se desplegó con Docker Compose y contempla cinco contenedores: Redis, Metrics Service, Response Service, Cache Service y Traffic Generator. El flujo es secuencial: tráfico $\rightarrow$ caché $\rightarrow$ respuestas, con registro transversal de métricas. Los servicios se coordinan mediante `healthcheck` y `depends_on` con `condition: service_healthy`, garantizando que el traffic generator no envíe solicitudes hasta que toda la cadena (descarga del dataset incluida) esté operativa.

2.2. Componentes Principales

2.2.1. Generador de Tráfico

Genera solicitudes sintéticas para Q1–Q5 bajo distribuciones uniforme y Zipf, para emular zonas calientes y patrones repetitivos.

2.2.2. Servicio de Caché

Implementado con Redis, soporta TTL y política de remoción configurable. En cada consulta registra hit/miss, latencia y evicciones.

2.2.3. Generador de Respuestas

Precarga el dataset de edificaciones de Google Open Buildings y calcula en memoria:

- Q1: conteo por zona y umbral de confianza.
- Q2: área promedio y área total.
- Q3: densidad por km^2 .

- Q4: comparación de densidad entre dos zonas.
- Q5: histograma de confianza.

2.2.4. Métricas

Se registran: hit rate, miss rate, throughput, latencias p50/p95, tasa de evicción y eficiencia de caché.

3. Dataset y Zonas de Consulta

3.1. Google Open Buildings v3

El dataset utilizado proviene de **Google Open Buildings**¹, un proyecto que contiene la ubicación, tamaño y nivel de confianza de edificaciones detectadas mediante imágenes satelitales. Se utiliza el archivo `967_buildings.csv.gz` (tile S2 level 4), que cubre la zona de Chile incluyendo la Región Metropolitana.

Los campos utilizados son:

Campo	Tipo	Descripción
<code>latitude</code>	FLOAT	Latitud del centroide
<code>longitude</code>	FLOAT	Longitud del centroide
<code>area_in_meters</code>	FLOAT	Área aproximada (m ²)
<code>confidence</code>	FLOAT	Confianza de detección [0,65; 1,0]

Tabla 1: Campos del dataset Google Open Buildings v3 utilizados.

3.2. Descarga automática y persistencia

Al iniciar, el `response-service` descarga el archivo comprimido desde GCS², lo extrae a la carpeta `/app/dataset/` y asigna cada registro a una de las cinco zonas predefinidas mediante sus bounding boxes. El archivo se almacena en un volumen Docker (`dataset:`) para persistir entre reinicios y evitar descargas repetidas. Si la velocidad promedio de descarga es inferior a 1 Mbps después de al menos 50 segundos, el servicio detiene la transferencia, arroja un error y permite reanudar desde el archivo parcial en los próximos reinicios de Docker, optimizando el consumo de red.

3.3. Zonas Geográficas Predefinidas

Se definen cinco zonas fijas de Santiago como bounding boxes, alineadas con el enunciado:

¹<https://sites.research.google/open-buildings/>

²https://storage.googleapis.com/open-buildings-data/v3/polygons_s2_level_4_gzip/967_buildings.csv.gz

Zona (ID)	lat_min	lat_max	lon_min	lon_max
Providencia (Z1)	-33,445	-33,420	-70,640	-70,600
Las Condes (Z2)	-33,420	-33,390	-70,600	-70,550
Maipú (Z3)	-33,530	-33,490	-70,790	-70,740
Santiago Centro (Z4)	-33,460	-33,430	-70,670	-70,630
Pudahuel (Z5)	-33,470	-33,430	-70,810	-70,760

Tabla 2: Bounding boxes de las cinco zonas de Santiago.

4. Implementación

- **FastAPI:** facilita APIs ligeras y validación de payloads.
- **Redis:** entrega operaciones $O(1)$, TTL nativo y políticas LRU/LFU/FIFO configurables.
- **Cálculo en memoria:** reduce complejidad operacional y cumple el requisito de no usar base de datos en runtime.
- **Healthchecks y orquestación:** cada servicio expone un endpoint `/health`. Docker Compose usa `condition: service_healthy` en las dependencias para garantizar que el traffic generator no envíe solicitudes hasta que el dataset esté cargado y todos los servicios respondan correctamente.

5. Experimentos y resultados

5.1. Metodología experimental

Los experimentos se ejecutaron sobre el sistema desplegado con `docker compose`, reiniciando el entorno entre escenarios para evitar contaminación de estado. Se usaron dos campañas:

- **Campaña A (alta cardinalidad):** 2 000 requests por escenario, TTL=300 s, variando distribución, política y tamaño de caché (50 MB, 200 MB y 500 MB).
- **Campaña B (sensibilidad TTL):** 2 000 requests por escenario, distribución Zipf, caché 200 MB, con 50 ms entre requests para forzar expiraciones y presionar la vigencia del TTL.

El procedimiento reproducible usado en todas las corridas fue el de usar el script `run_one_scenario.sh` que levanta el stack entero de Docker Compose inyectando el valor de memoria y políticas vía variables y comandos. Se verificaron manualmente endpoints como `/query` arrojando exitosamente respuestas para Q1 (conteo), Q2 (área), Q3 (densidad), Q4 (comparación densidad) y Q5 (distribución confianza).

5.2. Impacto de la distribución de tráfico

Distribución	Hit rate	Miss rate	Throughput (qps)	Latencia p95 (ms)
Uniforme	0.1970	0.8030	15.07	1101.44
Zipf	0.2235	0.7765	17.94	1443.65

Tabla 3: Comparación bajo política LRU y caché de 200 MB.

Con tráfico Zipf se observó una mayor localización de referencias, mejorando el hit rate en aproximadamente 2,6 puntos porcentuales y elevando el throughput (17.94 vs 15.07 qps). Esto se explica porque la distribución Zipf concentra las consultas en unas pocas zonas “calientes”, incrementando la probabilidad de encontrar la respuesta en caché. La latencia p95 de Zipf es superior debido a que la intensidad concentrada exacerba las demoras ante misses recurrentes que obligan a realizar recálculos iniciales en la misma sub-sección del dataset, sumado a factores de cola de la concurrencia.

5.3. Efecto de la política de remoción

Política	Hit rate	Throughput (qps)	Latencia p95 (ms)	Evictions/min
LRU	0.2618	16.06	898.73	528.59
LFU	0.2983	16.23	812.69	481.63
FIFO (aprox. con random)	0.2223	15.99	903.01	572.70

Tabla 4: Comparación bajo Zipf, caché de 200 MB, TTL=300 s.

A diferencia de estimaciones puramente analíticas, los datos empíricos arrojan un alto volumen de evicciones, demostrando que 200 MB exigen a Redis activar sus algoritmos de descarte. En este escenario intensivo Zipf, **LFU (Least Frequently Used) superó a las demás políticas**, obteniendo un Hit Rate de 29,83 % y el mayor Throughput (16.23 qps). LRU rindió un 26,18 %, mientras que Random/FIFO arrojó el peor hit rate (22,23 %), evidenciando que descartar aleatoriamente no permite que el sistema consolide un “working set” óptimo para una carga Zipf que penaliza fuertemente el desalojo de respuestas calientes.

5.4. Efecto del tamaño de caché

Tamaño	Hit rate	Miss rate	Throughput (qps)	Latencia p95 (ms)	Evictions/min
50 MB	0.0834	0.9166	12.68	978.88	632.70
200 MB	0.2607	0.7393	16.02	907.62	527.68
500 MB	0.4122	0.5878	18.94	839.80	401.03

Tabla 5: Sensibilidad del hit rate y rendimiento al tamaño de caché (LRU, Zipf, TTL=300 s).

Los resultados confirman que el rendimiento de la caché exhibe una alta dependencia de la memoria aprovisionada. Aumentar el tamaño de 50 MB a 500 MB multiplicó el Hit Rate ~ 5 veces (de 8.3 % a 41.22 %) y mejoró el throughput un 49 %. Las evicciones caen consistentemente a medida que la memoria crece, probando que 50 MB provoca “thrashing” extremo (632 evicciones por minuto) descartando contenido válido a una tasa inviable para sostener eficiencia general. Con 500 MB el impacto en p95 (cola larga) disminuye notablemente (839.8 ms).

5.5. Efecto de TTL

TTL (s)	Hit rate	Miss rate	Throughput (qps)	Latencia p95 (ms)
5	0.0403	0.9597	14.72	1454.72
20	0.1353	0.8647	14.96	1181.80
120	0.2155	0.7845	18.29	1359.97

Tabla 6: Resultados con 2 000 requests, Zipf, 200 MB y 50 ms de separación entre solicitudes.

Aumentar TTL reduce masivamente las expiraciones prematuras forzadas por el generador de 50 ms. Con TTL=5s el sistema apenas retiene aciertos (4 % Hit Rate) debido a que los items caducan antes de poder ser re-consumidos. Con 120 segundos, se quintuplica el rendimiento llegando a 21,55 % y empujando el qps hacia 18,29. Esto confirma que si los datos subyacentes son de baja rotación, estirar el TTL otorga ganancias directas en sistemas altamente distribuidos.

6. Discusión

6.1. Preguntas del Enunciado

- ¿Cómo varía la tasa de aciertos entre distribuciones? Zipf alcanza mejor Hit Rate (+2.6 p.p. vs Uniforme) y Throughput por la concentración de consultas en zonas preferentes, reduciendo escapes y maximizando utilidad de cache.

- **¿Qué distribución genera mayor estrés o beneficio para la caché?** Zipf beneficia el rendimiento global al incrementar la repetición temporal. Uniforme estresa el sistema barriendo uniformemente el espacio de llaves y generando el peor hit rate.
- **¿Qué implicaciones para un escenario real?** En escenarios como geolocalización o reparto, los patrones coinciden con Zipf (distribuciones de potencias/Pareto). Favorecer LFU y memorias amplias entrega resultados netos.
- **Políticas de remoción:** Se observó una victoria concreta de **LFU**. Descartar contenido por su baja frecuencia demostró acoplarse eficientemente con las descargas de Zipf, superando a LRU y dejando relegado a la variante aleatoria (FIFO).
- **Tamaño de caché:** En la campaña empírica fue un factor trascendental: incrementar de 50 a 500 MB quintuplicó el Hit Rate, sugiriendo que la huella de memoria necesaria es alta considerando overhead en Redis.
- **TTL:** Fundamental para retener datos valiosos entre tandas. Valores pequeños (5s) destrozan la eficiencia generando una degradación equivalente a carecer de memoria.

7. Conclusiones

Los resultados avalados por pruebas físicas refutan visiones estáticas demostrando que un diseño eficiente en redes distribuidas debe ajustarse fuertemente al análisis empírico. Se evidencia que una política **LFU** rinde sustancialmente mejor que una LRU bajo cargas en distribución Zipf, minimizando recálculos de áreas o conteos complejos en la memoria base de Google Open Buildings.

Paralelamente, la memoria (**Tamaño de caché**) probó dictar drásticos umbrales de sobrecarga, con tasas de evicción inmanejables a 50MB que cesan significativamente con incrementos a 500 MB. El TTL funciona como un factor multiplicativo; alinear un TTL corto es equivalente a un estrangulamiento de los recursos del caché. El sistema cumple todos los requerimientos y operó sobre tests controlados para resolver satisfactoriamente las consultas geográficas en memoria con el archivo parcial automatizado de reanudación.

8. Referencias

- Google Open Buildings Dataset: <https://sites.research.google/open-buildings/>
- Google Open Buildings v3 Data: https://storage.googleapis.com/open-buildings-data/v3/polygons_s2_level_4_gzip/967_buildings.csv.gz
- Redis Documentation: <https://redis.io/docs/>

Links

1. Repositorio de Github: <https://github.com/me1kimu/T1distribuido>
2. Video de funcionamiento: en Canvas.